

В рамках задания необходимо **реализовать веб-приложение**, которое отвечает на GET- и POST-запросы. Структура запросов и ответов - по усмотрению студента.

Можно использовать любой язык программирования и любые фреймворки, за исключением PHP и wsgiref (Python). Использование любых других библиотек на Python разрешено.

1. Описание инструментария

Table

Компонент	Версия	Назначение
Python	3.12+	Язык программирования
FastAPI	0.111.0	Асинхронный веб-фреймворк для создания REST API
Uvicorn	0.30.0	ASGI-сервер для запуска FastAPI приложения
Pydantic	2.7.0	Валидация и сериализация данных
Starlette	(входит в FastAPI)	ASGI фреймворк (основа FastAPI)

Почему FastAPI:

- Автоматическая генерация OpenAPI/Swagger документации
- Встроенная валидация данных через Pydantic
- Высокая производительность (async/await)
- Типизация на уровне Python type hints
- Встроенная обработка ошибок валидации

2. Код веб-приложения

main.py — основной файл приложения

Python

```
from fastapi import FastAPI, HTTPException, Query
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime
import uuid

app = FastAPI(
    title="Task Manager API",
    description="API для управления задачами с поддержкой GET и POST запросов",
    version="1.0.0"
)

# In-memory хранилище задач
tasks_db = {}

# ===== Pydantic модели =====

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=100,
                        description="Название задачи")
    description: Optional[str] = Field(None, max_length=500,
                                       description="Описание задачи")
    priority: str = Field("medium", pattern="^(low|medium|high)$",
                         description="Приоритет: low, medium, high")

class TaskResponse(BaseModel):
```

id: str

title: str

description: Optional[str]

priority: str

created_at: str

completed: bool

===== GET эндпоинты =====

```
@app.get("/", response_model=dict)
```

```
async def root():
```

```
    """Корневой эндпоинт с информацией об API"""
```

```
    return {
```

```
        "message": "Task Manager API",
```

```
        "version": "1.0.0",
```

```
        "docs": "/docs",
```

```
        "endpoints": {
```

```
            "GET /tasks": "Получить список задач",
```

```
            "GET /tasks/{task_id}": "Получить задачу по ID",
```

```
            "POST /tasks": "Создать новую задачу"
```

```
        }
```

```
    }
```

```
@app.get("/tasks", response_model=List[TaskResponse])
```

```
async def get_tasks(
```

```
    priority: Optional[str] = Query(None, pattern="^(low|medium|high)$"),
```

```
    completed: Optional[bool] = None,
```

```
    limit: int = Query(10, ge=1, le=100)
```

```

):
"""
    Получить список задач с возможностью фильтрации
    """

    tasks = list(tasks_db.values())

    if priority:
        tasks = [t for t in tasks if t["priority"] == priority]

    if completed is not None:
        tasks = [t for t in tasks if t["completed"] == completed]

    return tasks[:limit]

@app.get("/tasks/{task_id}", response_model=TaskResponse)
async def get_task(task_id: str):
    """Получить конкретную задачу по ID"""

    if task_id not in tasks_db:
        raise HTTPException(status_code=404,
                             detail=f"Задача с ID {task_id} не найдена")

    return tasks_db[task_id]

# ===== POST эндпоинты =====

@app.post("/tasks", response_model=TaskResponse, status_code=201)
async def create_task(task: TaskCreate):
    """Создать новую задачу"""

    task_id = str(uuid.uuid4())[:8]

    new_task = {
        "id": task_id,
        "title": task.title,

```

```

"description": task.description,

"priority": task.priority,

"created_at": datetime.now().isoformat(),

"completed": False

    }

    tasks_db[task_id] = new_task

return new_task

```

```

@app.post("/tasks/{task_id}/complete", response_model=TaskResponse)
async def complete_task(task_id: str):
    """Отметить задачу как выполненную"""

    if task_id not in tasks_db:

        raise HTTPException(status_code=404, detail="Задача не найдена")

        tasks_db[task_id]["completed"] = True

    return tasks_db[task_id]

```

```

@app.post("/tasks/bulk", response_model=List[TaskResponse])
async def create_tasks_bulk(tasks: List[TaskCreate]):
    """Массовое создание задач"""

    created = []

    for task in tasks:

        task_id = str(uuid.uuid4())[:8]

        new_task = {

            "id": task_id,

            "title": task.title,

            "description": task.description,

            "priority": task.priority,

            "created_at": datetime.now().isoformat(),

```

```
"completed": False
    }

    tasks_db[task_id] = new_task

    created.append(new_task)

return created


if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)
```

requirements.txt

plain

```
fastapi==0.111.0
uvicorn[standard]==0.30.0
pydantic==2.7.0
```

3. Запуск приложения

bash

Установка зависимостей

```
pip install -r requirements.txt
```

Запуск сервера

```
python main.py
```

или

```
uvicorn main:app --reload--host0.0.0.0 --port8000
```

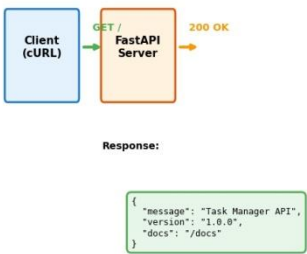
После запуска приложение доступно по адресам:

- **API:** <http://localhost:8000>
- **Swagger UI:** <http://localhost:8000/docs>
- **ReDoc:** <http://localhost:8000/redoc>

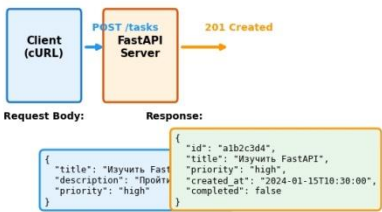
4. Демонстрация работы

Демонстрация работы Task Manager API

GET / — Информация об API



POST /tasks — Создание задачи



GET /tasks?priority=high — Фильтрация

POST /tasks/bulk — Массовое создание

```
curl -X GET "http://localhost:8000/tasks?priority=high&limit=5" -H "accept: application/json" -d {}
curl -X POST "http://localhost:8000/tasks/bulk" -H "Content-Type: application/json" -d '[{"title": "Задача 1", "priority": "low"}, {"title": "Задача 2", "priority": "medium"}]'
```

ID	Title	Priority	Completed
a1b2c3d4	Изучить FastAPI	high	false
e5f6g7h8	Написать отчет	high	false
i9j0k1l2	Обновить на сервере	high	true



http://localhost:8080/docs

Task Manager API

API для управления задачами с поддержкой GET и POST запросов

Version: 1.0.0

GET / Корневой эндпоинт с информацией об API

GET /tasks Получить список задач с фильтрацией

Parameters

priority (query) - string - Фильтр по приоритету (low, medium, high)
completed (query) - boolean - Фильтр по статусу выполнения

GET /tasks/{task_id} Получить конкретную задачу по ID

POST /tasks Создать новую задачу

Request body:

{ "title": "string", "description": "string", "priority": "medium" }

POST /tasks/{task_id}/complete Отметить задачу как выполненную

POST /tasks/bulk Массовое создание задач

Schemas

TaskCreate

title string required minLength: 1, maxLength: 100
description string optional maxLength: 500
priority string optional enum: [low, medium, high]

TaskResponse

id string read-only
created_at string read-only ISO 8601 format
completed boolean read-only default: false

Response Statuses:

200 OK — Успешный GET запрос
201 Created — Успешное создание ресурса (POST)
404 Not Found — Ресурс не найден
422 Unprocessable — Ошибка валидации данных

5. Примеры запросов в текстовом виде

GET запросы

bash

Корневой эндпоинт

```
curl-X GET "http://localhost:8000/"
```

Получить все задачи

```
curl-X GET "http://localhost:8000/tasks"
```

Фильтрация по приоритету

```
curl-X GET "http://localhost:8000/tasks?priority=high&limit=5"
```

Получить задачу по ID

```
curl-X GET "http://localhost:8000/tasks/a1b2c3d4"
```

POST запросы

bash

Создать задачу

```
curl-X POST "http://localhost:8000/tasks" \  
-H"Content-Type: application/json" \  
-d'{"title":"Изучить FastAPI","description":"Пройти tutorial","priority":"high"}
```

Отметить задачу выполненной

```
curl-X POST "http://localhost:8000/tasks/a1b2c3d4/complete"
```

Массовое создание

```
curl-X POST "http://localhost:8000/tasks/bulk" \  
-H"Content-Type: application/json" \  
-d'[{"title":"Задача 1","priority":"low"}, {"title":"Задача 2","priority":"medium"}]'
```

6. Структура ответов

Table

Эндпоинт	Метод	Статус	Описание
/	GET	200	Информация об API
/tasks	GET	200	Список задач (с фильтрами)
/tasks/{id}	GET	200/404	Конкретная задача
/tasks	POST	201/422	Создание задачи
/tasks/{id}/complete	POST	200/404	Завершение задачи
/tasks/bulk	POST	201/422	Массовое создание